

# FoSyMa Cours 4

## Protocoles de communication et consensus

Aurélie Beynier

FoSyMa, Master 1 ANDROIDE

3 février 2025

# Communication entre agents

Les protocoles doivent tenir compte des possibilités de communication entre agents.

Dans un environnement dynamique, la communication directe entre agents peut ne pas toujours être possible et garantir son maintien peut être coûteux (déplacement des agents en maintenant le réseau de communication par exemple).

Deux approches possibles :

- **établir un réseau de communication** lorsque les agents souhaitent communiquer et utiliser cette structure de communication (approche par echo-flooding par exemple)
- baser les communications sur des **interactions opportunistes paire-à-paire** lorsque les agents sont à proximité (approche par rumeur par exemple).

# Flooding

Flood protocol dans un réseau

---

**Algorithm 1:** Flooding protocol

---

La source envoie son message à tous ses voisins

Chaque voisin renvoie le message à tous ses voisins sauf celui dont il a reçu le message (parent), et envoie un message ack à son parent.

Lorsqu'un agent reçoit un message déjà reçu, il l'ignore et envoie un message *nack* à l'expéditeur

---

Construction d'un arbre couvrant dont le nœud racine est la source du message.

Cette arbre peut ensuite être réutilisé pour réduire les communications lors de la diffusion d'information (si la structure ne change pas).

Il est nécessaire de pouvoir identifier une information de manière unique.

# Flooding optimisé

Un message peut emprunter différents chemins pour arriver à un destinataire. On souhaite qu'il emprunte le plus court chemin.  
Chaque agent  $i$  maintient une distance  $d_i$  à la racine  $r$

---

## Algorithm 2: Echo flooding optimisé

---

Initialisation :  $d_r = 0$ , et  $d_i = \infty \forall i \neq r$

L'agent racine envoie la valeur 1 à tous ses voisins.

**if** *un agent reçoit la valeur  $v$ , telle que  $v < d_i$*  **then**

$d_i = v$

l'agent  $i$  envoie  $v + 1$  à tous ses voisins (sauf la source de  $v$ )

**end**

---

Algorithme de Bellman-Ford utilisé par le protocole RIP pour le routage dans les réseaux.

# Echo Flooding

S'accompagne d'une remontée d'information (une réponse par exemple) vers la racine

---

**Algorithm 3:** Echo Flooding protocol

---

La source envoie son message à tous ses voisins

Chaque voisin renvoie le message à tous ses voisins sauf celui dont il a reçu le message (parent), et envoie un message ack à son parent.

Lorsqu'un agent reçoit un message déjà reçu, il l'ignore et envoie un message *nack* à l'expéditeur

**Une fois qu'un agent  $i$  a reçu une réponse de tous ses enfants, il envoie un message à son parent**

---

# Diffusion d'informations dans un réseau

- Modèles de diffusion inspirés de la diffusion des maladies infectieuses ( modèles SIR et SIS) et des études en sociologie sur la diffusion dans les réseaux.
- Questions étudiées [Jac08] :
  - Est-ce que les agents vont adopter une opinion commune ou garder des opinions différentes ? un consensus va-t-il être atteint au sein de la population d'agents ? sous quelles conditions ?
  - Quels individus ont le plus d'influence sur la diffusion des opinions (leader dans la diffusion des opinions) ?
  - A quelle vitesse les opinions se diffusent-elles ?
  - Des informations disséminées dans la population peuvent-elles être agrégées de manière précise ?

# Gossip problem

- Un ensemble de  $n$  agents détenant chacun un morceau d'information (secret) incinué des autres.
- Les agents communiquent en interaction paire-à-paire lors de rencontre opportuniste par exemple ou en se téléphonant.
- Après un appel ou une rencontre, les deux agents ayant échangé connaissent tous les secrets de l'autre. Un agent connaissant tous les secrets est dit **expert**.
- Comment permettre à tous les agents de connaître tous les secrets ? Combien de rencontres / appels sont nécessaires ?
- Un protocole de Gossip précise comment se rencontrent les agents : quand et comment sélectionner les agents qu'on appelle ?

# Gossip problem

Exemple : 3 agents connaissant chacun initialement un unique secret :  $(A, B, C)$  [DKS17].

- $a$  appelle  $b$  :  $(AB, AB, C)$
- $b$  appelle  $c$  :  $(AB, ABC, ABC)$
- $a$  appelle  $c$  :  $(ABC, ABC, ABC)$  -

tous les agents sont experts



# Gossip problem - protocoles

Tant que tous les agents ne sont pas experts :

- **ANY** : sélectionner n'importe quels agents  $a$  et  $b$ ,  $a$  appelle  $b$ .
- **CO** : sélectionner deux agents  $a$  et  $b$  qui ne se sont pas encore rencontrés,  $a$  appelle  $b$ .
- **LNS** (Learn New Secret) : sélectionner deux agents  $a$  et  $b$  tels que  $a$  ne connaît pas le secret de  $b$ ,  $a$  appelle  $b$ .

*Remarque : la séquence de l'exemple précédent est permise par les 3 protocoles.*

Les protocoles dont le choix des appels dépendent des connaissances des agents sont appelés *protocoles de Gossip épistémiques*

# Gossip problem - protocoles

En supposant que chaque agent peut appeler tous les autres agents.

## Borne supérieure sur le nombre d'appels

- ANY : la longueur de la plus longue séquence d'appels est infinie .
- CO : la longueur de la plus longue séquence d'appel est  $\binom{n}{2}$
- LNS : la longueur de la plus longue séquence d'appels est  $\binom{n}{2}$

# Gossip problem - protocoles

En supposant que chaque agent peut appeler tous les autres agents.

## Atteignabilité (Reachability)

$n$  agents peuvent devenir experts en  $\lceil \log_2 n \rceil$  appels si  $n$  est pair, et  $\lceil \log_2 n \rceil + 1$  appels si  $n$  est impair.

## Borne inférieure sur le nombre d'appels

Si les  $n$  agents peuvent se joindre, le nombre minimum d'appels pour que tous les agents soient experts (pour  $n \geq 4$ ) est  $2n - 4$ .

# Protocole Push-Sum [KDG03]

- Agrégation de valeurs dans un réseau : chaque agent détient une valeur.
- On souhaite calculer la somme, la moyenne de ces valeurs.
- Chaque agent maintient une valeur et un poids en fonction des messages reçus.
- A chaque tour  $t$ , chaque agent (ou nœud) calcule une somme  $s_{t,i}$  et un poids  $w_{t,i}$  en fonction des messages reçus.
- La moitié de la somme de ses poids et ses valeurs est ensuite envoyée à un autre agent choisi au hasard (et à soi même )

# Protocole Push-Sum

---

## Algorithm 4: Push-Sum

---

Initialisation :  $s_{0,i} = x_i$ , et  $w_{0,i} = 1$ .

A chaque tour  $t$

Soit  $\{(\hat{s}_r, \hat{w}_r)\}$  toutes les paires reçues par  $i$  au tour  $t - 1$

$$s_{t,i} = \sum_r \hat{s}_r, w_{t,i} = \sum_r \hat{w}_r$$

Sélectionner un destinataire  $f_t(i)$  uniformément au hasard

Envoyer la paire  $(\frac{1}{2}s_{t,i}, \frac{1}{2}w_{t,i})$  à  $f_t(i)$  et à  $i$  (soi-même)

$\frac{s_{t,i}}{w_{t,i}}$  est une approximation de la moyenne à l'étape  $t$

---

Le protocole Push-Sum converge vers une “bonne” estimation de la moyenne en  $O(\log n)$  tours

# Diffusion d'information, consensus [Jac08]

- Modèle de diffusion d'information sur les réseaux proposé par DeGroot [DeG74]
- Les agents possèdent initialement des opinions différentes sur un même sujet représentées par un vecteur de probabilités

$$p(0) = (p_1(0), \dots, p_n(0)), \text{ avec } p_i(0) \in [0, 1]$$

où  $p_i$  est, du point de vue de l'agent  $i$ , la probabilité que le fait considéré soit vrai.

- Les interactions entre agents sont représentées par une matrice  $T$  de taille  $n \times n$  telle que la somme des valeurs de chaque ligne est de 1.  $T_{ij}$  est le poids (ou confiance) que l'agent  $i$  place dans l'agent  $j$ .
- La mise à jour des croyances à l'instance  $t$  se fait à partir de  $T$  et  $p(t-1)$  :

$$p(t) = Tp(t-1) = T^t p(0)$$

# Diffusion d'information, consensus

Exemple :

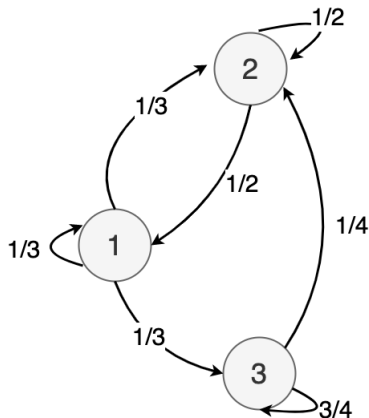
$$T = \begin{pmatrix} 1/3 & 1/3 & 1/3 \\ 1/2 & 1/2 & 0 \\ 0 & 1/4 & 3/4 \end{pmatrix}$$

et

$$p(0) = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

# Diffusion d'information, consensus

Exemple :





# Diffusion d'information, consensus

Exemple :

$$p(1) = Tp(0) = \begin{pmatrix} 1/3 & 1/3 & 1/3 \\ 1/2 & 1/2 & 0 \\ 0 & 1/4 & 3/4 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1/3 \\ 1/2 \\ 0 \end{pmatrix}$$

$$p(2) = Tp(1) = \begin{pmatrix} 1/3 & 1/3 & 1/3 \\ 1/2 & 1/2 & 0 \\ 0 & 1/4 & 3/4 \end{pmatrix} \begin{pmatrix} 1/3 \\ 1/2 \\ 0 \end{pmatrix} = \begin{pmatrix} 5/18 \\ 5/12 \\ 1/8 \end{pmatrix}$$

⋮

$$p(t) = Tp(t-1) = T^t p(0) \rightarrow \begin{pmatrix} 3/11 \\ 3/11 \\ 3/11 \end{pmatrix}$$

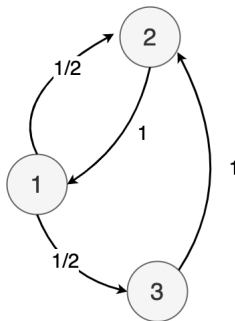
# Diffusion d'information, consensus

- La matrice  $T$  permet de représenter des connexions dans un réseau social.
- Le modèle permet également de modéliser des agents  $i$  diffuseurs d'opinions, très écoutés des autres ( $T_{ji}$  proche de 1) et peu ou pas influencés par les opinions des autres ( $T_{ij} = 0$ ). L'opinion de ces agents  $i$  reste à  $p_i(0)$ .
- La convergence vers un état stable est-elle garantie ? la convergence se fait-elle toujours vers un état de consensus ?

# Diffusion d'information, consensus

## Exemple 2

$$T = \begin{pmatrix} 0 & 1/2 & 1/2 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$



# Diffusion d'information, consensus

Exemple :

$$T^2 = \begin{pmatrix} 1/2 & 1/2 & 0 \\ 0 & 1/2 & 1/2 \\ 1 & 0 & 0 \end{pmatrix}, T^3 = \begin{pmatrix} 1/2 & 1/2 & 1/4 \\ 1/2 & 1/2 & 0 \\ 0 & 1/2 & 1/2 \end{pmatrix}, T^4 = \begin{pmatrix} 1/4 & 1/2 & 1/4 \\ 1/2 & 1/2 & 1/4 \\ 1/2 & 1/2 & 0 \end{pmatrix}$$

$$T^t = \begin{pmatrix} 2/5 & 2/5 & 1/5 \\ 2/5 & 2/5 & 1/5 \\ 2/5 & 2/5 & 1/5 \end{pmatrix}$$

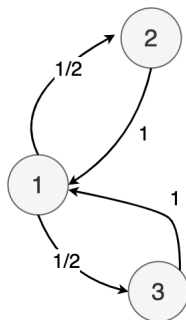
Quelque soit  $p(0)$ , les agents vont atteindre un consensus tel que :

$$p_1(\infty) = p_2(\infty) = p_3(\infty)n = \frac{2}{5}p_1(0) + \frac{2}{5}p_2(0) + \frac{1}{5}p_3(0)$$

# Diffusion d'information, consensus

## Exemple 2 - variation

$$T = \begin{pmatrix} 0 & 1/2 & 1/2 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$



# Diffusion d'information, consensus

Exemple variation :

$$T^2 = \begin{pmatrix} 1 & 10 & 0 \\ 0 & 1/2 & 1/2 \\ 0 & 1/2 & 1/2 \end{pmatrix}, T^3 = \begin{pmatrix} 1/2 & 1/2 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}, T^4 = \begin{pmatrix} 1 & 10 & 0 \\ 0 & 1/2 & 1/2 \\ 0 & 1/2 & 1/2 \end{pmatrix}, \dots$$

Oscillation, pas de convergence

Il existe un cycle dans le réseau de mise à jour et tous les cycles ont une longueur multiple de 2  $\Rightarrow T$  est dite **periodique**

# Diffusion d'information, consensus

Considérons le graphe orienté issu de  $T$  :

- $T$  est dit **apériodique** si le plus grand diviseur commun des longueurs de tous les cycles orientés est 1. ( $T_{ii} > 0$  pour au moins un agent garantit la **apériodicité**).
- Un groupe d'agents est **fortement connecté** si il existe un chemin orienté de n'importe quel agent à n'importe quel autre agent dans le groupe.
- Un ensemble  $C$  d'agents est **clos** s'il n'existe pas de lien direct entre un agent de  $C$  et un agent en dehors de  $C$

## Garantie de convergence

$T$  est convergente si et seulement si tout ensemble de nœuds fortement connecté et clos est apériodique.

# Diffusion d'information, consensus

Considérons le graphe orienté issu de  $T$  :

- $T$  est dit **apériodique** si le plus grand diviseur commun des longueurs de tous les cycles orientés est 1. (Remarque :  $T_{ii} > 0$  pour au moins un agent garantit la **apériodicité**.)
- Un groupe d'agents est **fortement connecté** si il existe un chemin orienté de n'importe quel agent à n'importe quel autre agent dans le groupe.
- Un ensemble  $C$  d'agents est **clos** s'il n'existe pas de lien direct entre un agent de  $C$  et un agent en dehors de  $C$

## Garantie de consensus

Un consensus est atteint dans le modèle de DeGRoot si et seulement si il existe exactement un groupe d'agents fortement connecté et clos et  $T$  est apériodique sur ce groupe.



# Références I



Morris H. DeGroot, *Reaching a consensus*, Journal of the American Statistical Association **69** (1974), no. 345, 118–121.



Hans Ditmarsch, Ioannis Kokkinis, and Anders Stockmarr, *Reachability and expectation in gossiping*, 10 2017.



Matthew O. Jackson, *Social and economic networks*, Princeton University Press, Princeton, NJ, 2008.



D. Kempe, A. Dobra, and J. Gehrke, *Gossip-based computation of aggregate information*, 44th Annual IEEE Symposium on Foundations of Computer Science, 2003. Proceedings., 2003, pp. 482–491.